

مشروع Colab — الأكواد الكاملة

النسخة الكاملة من جميع خلايا Colab

الإصدار 1.0 — سبتمبر 2026

جميع خلايا Colab

خلية 0 - تثبيت المكتبات

```
!pip install -q playwright pandas openpyxl xlswriter nest-asyncio
!playwright install chromium
!apt-get install -y libnss3 libatk-bridge2.0-0 libcups2 libdrm2 \
  libxkbcommon0 libxcomposite1 libxdamage1 libxfixes3 libxrandr2 \
  libgbm1 libpango-1.0-0 libcairo2
```

سحب الأسهم - Aخلية 1

```

import asyncio
import pandas as pd
from playwright.async_api import async_playwright

SOURCES = ["مَصْفَى", "مَصْفَى", "كاشف", "بنك فيصل", "أسطول",
            "Islamicly / Thndr", "جروب الأسهم الشرعية فقط"]
MIN_GREEN = 3

async def fetch_stocks():
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True,
            args=['--no-sandbox', '--disable-setuid-sandbox',
                '--disable-dev-shm-usage', '--disable-gpu'])
        ctx = await browser.new_context(locale="ar-EG")
        page = await ctx.new_page()
        await page.goto("https://stocks.templatesnippet.com/stocks",
            timeout=60000, wait_until="networkidle")
        await asyncio.sleep(3)
        try:
            await page.click("text=متابعة, فهمت", timeout=3000)
            await asyncio.sleep(1)
        except: pass
        for i in range(60):
            clicked = await page.evaluate("""() => {
                const btns = Array.from(document.querySelectorAll('button, a'));
                const btn = btns.find(b => (b.innerText||'').includes('تحميل المزيد'));
                if (!btn) return 'DONE';
                btn.scrollIntoView({block:'center'});
                btn.click();
                return 'CLICKED';
            }""")
            if clicked == 'DONE': break
            await asyncio.sleep(1.8)
        js = '''
        (sources) => {
            const MIN = 3;
            const results = [];
            const cards = document.querySelectorAll('div.group.rounded-2xl.border');
            cards.forEach(card => {
                const greenBadges = card.querySelectorAll(
                    'span[class*="bg-green-"], span[class*="text-green-"]');
                const green = new Set();
                greenBadges.forEach(b => {
                    const t = (b.innerText||'').trim();
                    if (sources.includes(t)) green.add(t);
                });
                if (green.size < MIN) return;
                const rawLines = (card.innerText||'').split(/\r?\n/)
                    .map(l => l.trim()).filter(l => l.length>0);
                let name = "غير معروف", code = "", category = "";
                for (const ln of rawLines) {
                    if (sources.includes(ln)) continue;
                    if (ln.includes("جروب الأسهم")) continue;
                    if (/^[A-Z]{2,6}$/.test(ln) && !code) { code = ln; continue; }
                    if (!category && ln.length > 2) { category = ln; continue; }
                    if (name === "غير معروف") name = ln;
                }
                results.push({
                    code: code||'-' , name: name, category: category||'-' ,
                    green_count: green.size,
                    green_sources: Array.from(green).join(" , ")
                });
            });
            return results;
        }'''
        data = await page.evaluate(js, SOURCES)
        await browser.close()
        return pd.DataFrame(data).rename(columns={
            "code": "الرمز", "name": "اسم السهم", "category": "التصنيف",
            "green_count": "عدد المصادر الخضراء",
            "green_sources": "المصادر المتفقة"
        })

```

```

})

df_result = await fetch_stocks()
print(f"✅ {len(df_result)} سهم")

```

1 سحب – بخلية Mubasher

```

import asyncio
import pandas as pd
from playwright.async_api import async_playwright

async def fetch_mubasher(page, code):
    try:
        await page.goto(f"https://www.mubasher.info/markets/EGX/stocks/{code}",
                        timeout=30000, wait_until="domcontentloaded")
        await asyncio.sleep(2.5)
        return await page.evaluate("""() => {
            const priceEl = document.querySelector('.market-summary__last-price');
            const price = priceEl ? priceEl.innerText.trim() : null;
            const text = document.body.innerText;
            const updM = text.match(/آخر تحديث[:\s]*([^\n]+)/);
            const prevM = text.match(/إغلاق سابق\s*([^\d.,]+)/);
            const openM = text.match(/فتح\s*([^\d.,]+)/);
            return {price,
                    lastUpdate: updM ? updM[1].trim() : null,
                    prevClose: prevM ? prevM[1] : null,
                    open: openM ? openM[1] : null};
        }""")
    except: return {"price": None}

async def fetch_all(df_result):
    codes = df_result["الرمز"].astype(str).tolist()
    results = []
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True,
            args=['--no-sandbox', '--disable-dev-shm-usage'])
        ctx = await browser.new_context(locale="ar-EG")
        page = await ctx.new_page()
        for i, code in enumerate(codes, 1):
            print(f"[{i}/{len(codes)}] {code}")
            d = await fetch_mubasher(page, code)
            d["الرمز"] = code
            results.append(d)
        await browser.close()
    return pd.DataFrame(results)

df_mubasher = await fetch_all(df_result)

```

1 سحب – بخلية EGXBot

```

import asyncio
import pandas as pd
from playwright.async_api import async_playwright

async def fetch_egxbot(page, code):
    try:
        await page.goto(f"https://egxbot.com/stock/{code}",
                        timeout=30000, wait_until="domcontentloaded")
        await asyncio.sleep(2.5)
        return await page.evaluate("""() => {
            const text = document.body.innerText;
            const r = {};
            const priceM = text.match(/([\d,]+(?:\.\d+)?)\s*جنيه/);
            if (priceM) r.price = parseFloat(priceM[1].replace(/,/g, ''));
            const chgM = text.match(/([+-]?\d+(?:\.\d+)?)%/);
            if (chgM) r.change = parseFloat(chgM[1]);
            const updM = text.match(/آخر تحديث[:\s]*([\d\-:\s]+)/);
            if (updM) r.last_update = updM[1].trim();
            const ratM = text.match(/(\d+)\s*من\s*100/);
            if (ratM) r.rating = parseInt(ratM[1]);
            return r;
        }""")
    except: return {"error": True}

async def fetch_all(df_result):
    codes = df_result["الرمز"].astype(str).tolist()
    results = []
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True,
                                           args=['--no-sandbox', '--disable-dev-shm-usage'])
        ctx = await browser.new_context(locale="ar-EG")
        page = await ctx.new_page()
        for i, code in enumerate(codes, 1):
            print(f"[{i}/{len(codes)}] {code}")
            d = await fetch_egxbot(page, code)
            d["الرمز"] = code
            results.append(d)
        await browser.close()
    return pd.DataFrame(results)

df_egxbot = await fetch_all(df_result)

```

```

!pip install -q pdfplumber

import pdfplumber
import pandas as pd
import re

pdf_path = "13-9-2026.pdf"
all_rows = []

with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        tables = page.extract_tables()
        for table in tables:
            for row in table:
                all_rows.append(row)

stocks_data = []
for row in all_rows:
    if not row or len(row) < 13: continue
    code = None; code_idx = None
    for i, cell in enumerate(row):
        if not cell: continue
        if re.match(r'^[A-Z]{2,6}$', str(cell).strip()):
            code = str(cell).strip(); code_idx = i; break
    if not code: continue
    try:
        nums = row[code_idx+1:]
        def to_num(x):
            if not x: return None
            try: return float(str(x).replace(",",""))
            except: return None
        v = [to_num(c) for c in nums]
        non_null = [x for x in v if x is not None]
        if len(non_null) < 9: continue
        stocks_data.append({
            "الرمز": code,
            "المقاومة الأولى": v[0], "المقاومة الثانية": v[1],
            "المقاومة الثالثة": v[2], "الدعم الأول": v[3],
            "الدعم الثاني": v[4], "الدعم الثالث": v[5],
            "ارتكاز": v[6], "التغير": v[7], "% التغير": v[8],
        })
    except: pass

df_support = pd.DataFrame(stocks_data).drop_duplicates("الرمز")

```

```

import pandas as pd
import re

df = df_result.merge(df_mubasher, on="الرمز", how="left", suffixes=("", "_p"))
df = df.merge(df_egxbot, on="الرمز", how="left", suffixes=("", "_e"))
df = df.loc[:, ~df.columns.duplicated()]

def convert_to_kmb(text):
    if not text or pd.isna(text): return text
    text = str(text).strip()
    for pattern, suffix in [
        (r'([\d.,]+)\s*مليار', 'B'),
        (r'([\d.,]+)\s*مليون', 'M'),
        (r'([\d.,]+)\s*ألف', 'K'),
    ]:
        m = re.search(pattern, text)
        if m: return f"{m.group(1)}{suffix}"
    return text

for col in ["حجم التداول", "القيمة السوقية"]:
    if col in df.columns:
        df[col] = df[col].apply(convert_to_kmb)

df.to_pickle("df_final_complete.pkl")

```

Excel خلية 4 – بناء

```

import pandas as pd
import matplotlib.pyplot as plt
from openpyxl import load_workbook
from openpyxl.styles import Font, PatternFill

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
plt.savefig("dashboard_charts.png", dpi=100)
plt.close()

OUTPUT_FILE = "stocks_analysis_final.xlsx"
with pd.ExcelWriter(OUTPUT_FILE, engine="xlsxwriter") as writer:
    df.to_excel(writer, sheet_name="البيانات الكاملة", index=False)
    df[df["عدد المصادر الخضراء"]==6].to_excel(
        writer, sheet_name="الأسهم النقية", index=False)

wb = load_workbook(OUTPUT_FILE)
wb.save(OUTPUT_FILE)

```

Drive خلية 5 – الحفظ على

```
from google.colab import drive
drive.mount('/content/drive')
import shutil, os

DRIVE = "/content/drive/MyDrive/Sharayia_stocks-github"
os.makedirs(DRIVE, exist_ok=True)

for f in ["df_result.csv", "df_final_complete.csv",
         "stocks_analysis_final.xlsx", "dashboard_charts.png"]:
    if os.path.exists(f):
        shutil.copy(f, DRIVE)

print("✅ تم الحفظ على Drive")
```